Workbench

Setup & usage guide — write it once, follow it cold

This walks through everything needed to deploy Workbench from a completely fresh checkout, install the one short command you'll actually type day to day, write and publish posts, and redo the setup on a different computer. It assumes no memory of how the project was built — every command is written out in full.

What this project is: the public site (home page, individual posts, search, tags, RSS, comments) is a plain static site with no backend, login, or database — anyone can view it, nobody but you can write to it. Writing happens through a separate local-only tool that never leaves your machine, so there's no password to manage and nothing to secure on the public side.

---

1. First-time deploy, from a fresh checkout

1.1 — Install Go

You need Go 1.22 or newer. Check what you have:

```
go version
```

If it's missing or older, install the current version from https://go.dev/dl/.

1.2 — Install the global `workbench` command (one-time, optional)

This step is optional — `go run . write` / `go run . build` from inside the repo always work with no setup. But a one-time install gives you a short `workbench` command you can run from any directory, with no `cd` and no recompiling each time. From inside the repo:

```
go install .
```

This builds a `workbench` binary into your Go bin directory (`go env GOPATH` + `/bin`, or `GOBIN` if you've set it — make sure that directory is on your `PATH`). Then tell it where this repo lives, once, by adding one line to your shell's rc file (`~/.bashrc`, `~/.zshrc`, etc.):

```
echo 'export WORKBENCH_REPO="$(pwd)"' >> ~/.bashrc
source ~/.bashrc
```

Replace `~/.bashrc` with your shell's rc file if you're not on bash. From then on, `workbench write` and `workbench build` work from any terminal, in any directory — the binary switches into `WORKBENCH_REPO` before it does anything else, so it still finds `/posts`, `site.url` and `giscus.json` in the right place. Re-run `go install .` any time the code in this repo changes, to refresh the installed binary.

1.3 — Get the code

If you already have a GitHub repo for this project, clone it:

```
git clone https://github.com/<your-username>/workbench.git
cd workbench
```

If you're starting from this project directory and it isn't pushed anywhere yet, create the repo and push in one go (requires the GitHub CLI, `gh`, already logged in via `gh auth login`):

```
gh repo create workbench --public --source=. --remote=origin
git push -u origin main
```

GitHub Pages' free tier requires the repo to be public (a private repo needs GitHub Pro/Team). If that's not acceptable, the alternative is a small VPS or a platform like Railway/Fly.io running `go run . build` on a schedule or webhook — not covered here since the static-site path above is simpler and free.

1.4 — Turn on GitHub Pages (one-time, per repository)

1. On github.com, open the repo → **Settings** → **Pages** (left sidebar, under "Code and automation").
2. Under **Build and deployment** → **Source**, choose **GitHub Actions** (not "Deploy from a branch"). Do this before your first push, or the very first workflow run can fail looking for a Pages site that doesn't exist yet.

That's the only manual setting. The repo already includes `.github/workflows/deploy.yml`, which runs on every push to `main`: it checks out the code, runs `go run . build`, and uploads the resulting `./public` directory as the Pages deployment. Nothing else to configure.

1.5 — Push and verify it went live

```
git push origin main
```

1. On github.com, open the **Actions** tab and watch the "Deploy Workbench" run finish (green check, usually under a minute).
2. Back in **Settings** → **Pages**, the live URL is shown at the top (something like https://<username>.github.io/workbench/).
3. Open that URL. If there are no posts yet, the home page will legitimately be empty — that's correct, not broken.

1.6 — Optional: a correct RSS feed URL

The RSS feed works without this, but item links will be relative instead of absolute unless you add a one-line file at the repo root:

```
echo "https://<username>.github.io/workbench" > site.url
git add site.url && git commit -m "set site URL" && git push
```

1.7 — One-time comments setup (giscus)

Comments are powered by giscus, backed by GitHub Discussions on this same repo. No server code needed on your end, but three things need doing once:

1. In the repo: **Settings** → **General** → **Features** → turn on **Discussions**.
2. Install the giscus GitHub App on this repo: visit https://github.com/apps/giscus and grant it access.
3. Visit https://giscus.app, enter your repo name, and it will generate four values: `repo`, `repoId`, `category`, `categoryId`.
4. Create a file named `giscus.json` at the repo root with those four values (see below), then commit and push.

```
{
  "repo": "your-username/workbench",
  "repoId": "...from giscus.app...",
  "category": "General",
  "categoryId": "...from giscus.app..."
}
```

```
git add giscus.json
git commit -m "enable comments"
git push
```

Until this file exists with all four fields filled in, the build simply omits the comments section from every post — no broken widget, no error box, nothing visible. It appears automatically on the next deploy once the file is committed.

2. Writing, day to day

All writing happens through a small local tool that only listens on `127.0.0.1` (your own machine) — nothing outside your computer can ever reach it, which is why there's no login for it.

2.1 — Start it

```
workbench write
# or, from inside the repo: go run . write
# or on a different port:            workbench write 9000
```

(The `workbench` form needs the one-time install from step 1.2; `go run .` always works with no install, from inside the repo directory.) Then open http://localhost:8090 (or whatever port you used) in your browser.

2.2 — Write a new post

1. Click **+ New post**.
2. Fill in the title, comma-separated tags, and date (defaults to today).

3. Write in the left pane in Markdown. The right pane shows a live preview, rendered through the exact same template and styling as the published post page — what you see there is what a visitor will see.
4. That's it — there's no Save button to click. The page autosaves about a second after you stop typing, and the status line at the top right shows *saving…* / *saved* / *unsaved changes*.

A footnote reference like `[^1]` in the text, paired with `[^1]: the note text` anywhere else in the body, becomes a margin sidenote on the published post.

2.3 — Edit an existing post

Go back to the post list (the Workbench link, top left) and click the post's title. It loads into the same editor, fully pre-filled, and autosaves the same way. The post's URL slug — set once when the post is first created from its title — never changes afterward, even if you edit the title later. That keeps links and comment threads stable.

2.4 — Stop it

Press `Ctrl+C` in the terminal where it's running. It's not a background service — when you're not actively writing, nothing related to the writer is running at all.

3. Publishing after you've written something

"Publishing" just means: the Markdown file you wrote is committed and pushed, which makes GitHub rebuild and redeploy the static site automatically. There's no separate build or deploy command to run by hand.

From "I just saved a draft" to "it's live":

1. (Optional) Stop the writer with `Ctrl+C` if it's still running.
2. (Optional) Preview exactly what will ship, locally: run `workbench build` (or `go run . build`), then open `public/index.html` in a browser. This writes to `./public`, which is git-ignored and not part of what gets committed.
3. Stage and commit the new or changed file(s) in `/posts`:

```
git add posts/
git commit -m "add: <post title>"
```

4. Push:

```
git push
```

That push alone triggers the "Deploy Workbench" GitHub Action: it runs `go run . build` fresh on GitHub's servers and deploys the result. Check the Actions tab for the green check, then reload the live site — usually live within a minute of pushing.

There is no draft/published toggle. A post is published the moment its file exists in `/posts` and that commit is pushed to `main`. If you want to hold a post back, just don't commit/push it yet — it can sit locally, half-written, for as long as you like.

4. Setting this up on a different computer

Short version: install Go, clone the repo, done. Specifically:

1. Install Go 1.22+ (section 1.1).
2. `git clone https://github.com/<you>/workbench.git && cd workbench` — this brings every post, the templates, and this PDF along with it, since they're all committed to the repo.
3. `go run . write` works immediately — no environment variables, no password, no database file, no config to copy over. That's the main simplification versus the old admin-login version of this project: there is no secret state tied to any one machine.
4. `go run . build` also works immediately, for a local preview.
5. If you want the global `workbench` command on this machine too, repeat step 1.2 here: `go install .`, then export `WORKBENCH_REPO` pointing at this clone's path in your shell rc file. This is the only step that's genuinely per-machine.

Nothing to redo on the GitHub side — Pages, the Actions workflow, and the giscus configuration are all properties of the *repository*, not of any particular computer. Once they're set up once, pushing from any machine (or several) triggers the same deploy.

The only genuinely per-machine things are having Go installed, the repo cloned, and (if you want the short command) the one-time `go install` + `WORKBENCH_REPO` export above — there is no local-only state (no password hash, no database, no session) left over from the previous admin-based version of this project to carry over or worry about losing.